

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**  
**KHOA CÔNG NGHỆ THÔNG TIN 1**



**MAJOR ASSIGNMENT REPORT DATABASE MANAGEMENT SYSTEM**

**BOARDING HOUSE MANAGEMENT SYSTEM**

|                 |   |              |            |
|-----------------|---|--------------|------------|
| Class           | : | D23CQCE06-B  |            |
| Group           | : | 01           |            |
| Students and ID | : | Phạm Lê An   | B23DCCE003 |
|                 |   | Vũ Gia Khánh | B23DCCE054 |

**HÀ NỘI – 2026**

# TABLE OF CONTENTS

|                                        |    |
|----------------------------------------|----|
| PROBLEM INTRODUCTION.....              | 3  |
| CONFIGURATION,TOOLS,AND PLATFORMS..... | 7  |
| DATABASE DESIGN OVERVIEW.....          | 10 |
| IMPLEMENTATION AND DEPLOYMENT .....    | 14 |
| CONCLUSION .....                       | 24 |
| REFERENCES.....                        | 28 |

# PROBLEM INTRODUCTION

## I. Problem Statement

In the current context of urban development, the demand for rental housing is surging. However, many boarding house owners, particularly the elderly, still rely on traditional manual management methods, such as paper notebooks or basic spreadsheets. This approach leads to several critical issues:

- **Data Inconsistency:** Manual entry often results in errors regarding tenant information, contract durations, and payment histories.
- **Complex Billing:** Calculating monthly bills that combine fixed room rates with fluctuating utility costs (electricity, water, services) is time-consuming and prone to miscalculation.
- **Debt Management:** Tracking outstanding balances and partial payments across multiple billing cycles is difficult without an automated system.
- **Lack of Transparency:** Tenants may find it hard to verify their service usage or past payment records, leading to potential disputes.

Therefore, there is an urgent need for a robust **Boarding House Management System** that provides an intuitive interface, ensures data integrity through automated triggers and transactions, and simplifies financial tracking for landlords.

## II. Project Objectives

The primary objective of this project is to develop a robust and user-friendly Boarding House Management System tailored for elderly landlords. The specific goals include:

- **System Automation:** To automate the calculation of monthly invoices by integrating room rates and variable service costs, reducing manual workload.
- **Data Integrity and Security:** To ensure high data reliability using MySQL constraints, triggers, and transactions, preventing loss or inconsistency during payment processing.
- **Efficient Debt Tracking:** To provide a real-time monitoring system for outstanding balances and partial payments through automated views and reports.
- **User-Centric Design:** To design a simplified interface that allows elderly users to manage contracts and view financial status with minimal technical effort.
- **Historical Accuracy:** To maintain a precise history of price adjustments and contract modifications without affecting previous financial records.

### III. Project Scope

- **Tenant Management:** Storing personal details, multiple contact numbers, and rental history.
- **Room and Contract Management:** Handling room availability, contract lifecycles, and automated deposit tracking.
- **Service and Billing:** Managing mandatory and optional services; automating monthly invoice generation for room rent and utilities.
- **Financial Monitoring:** Recording partial payments, tracking outstanding debts, and generating revenue reports via database views.
- **Data Integrity:** Utilizing Triggers and Transactions to ensure consistent financial records

## IV. Project Limitations:

- **Manual Data Entry:** The system currently requires manual input for electricity and water consumption (no IoT integration for smart meters).
- **Platform Constraint:** The current version is designed as a standalone desktop application (WPF/C#) and does not support mobile access or cloud-based synchronization.
- **Payment Gateway:** The system tracks payments recorded by the landlord but does not yet integrate with online payment gateways like MoMo or VNPay.

# CONFIGURATION, TOOLS, AND PLATFORMS

## I. Database Management System

For this project, **MySQL** has been selected as the primary Database Management System. The decision is based on several technical advantages that align with the requirements of a boarding house management application:

- **Relational Model:** MySQL effectively handles complex relationships between tenants, rooms, and contracts through its robust support for Primary and Foreign Keys.
- **ACID Compliance:** By utilizing the **InnoDB** storage engine, the system ensures Atomicity, Consistency, Isolation, and Durability. This is crucial for financial transactions where payment records must be accurate and reliable.
- **Performance and Scalability:** MySQL provides high-speed data processing, which is essential for generating real-time reports and handling concurrent data access as the number of rooms increases.
- **Advanced Logic Support:** It offers comprehensive support for **Stored Procedures, Triggers, and Views**, allowing complex business logic (like automated billing) to be handled directly within the database layer.
- **Ease of Integration:** MySQL integrates seamlessly with the **.NET framework** and **C#**, providing a stable backend for the WPF desktop application.

## II. Development Environment

To ensure the system's stability and performance, the following development environment was utilized:

- **Integrated Development Environment (IDE):**
  - **Microsoft Visual Studio 2022:** Used as the primary IDE for developing the desktop application using C# and WPF.
  - **MySQL Workbench :** Used for designing the EER diagrams, executing SQL scripts, and managing the database schema.
- **Programming Languages & Frameworks:**
  - **C# (.NET Framework/Core):** The core programming language used for building the application logic.
  - **Windows Presentation Foundation (WPF):** The framework used for designing a modern, user-friendly graphical interface (GUI).
  - **SQL (Structured Query Language):** Used for database definition, manipulation, and implementing advanced logic like Triggers and Procedures.
- **Database Management System:**
  - **MySQL Server :** The primary database engine chosen for its reliability and support for transactional integrity via the InnoDB engine.
- **Documentation Tools:**
  - **Microsoft Word:** Used for professional report formatting and academic documentation



### III. Hardware Configuration For Development And Testing

The following hardware specifications represent the environment used for the development, debugging, and local testing of the project:

- **Processor (CPU):** Intel(R) Core(TM) i7-14650HX( 24 CPUs ),~2.2GHz - Providing sufficient processing power to handle local database transactions and UI rendering.
- **Memory (RAM):** 16GB — Essential for running heavy development tools like **Visual Studio 2022** alongside the MySQL service without lagging.
- **Storage:** 1TB SSD — Ensuring fast read/write speeds for database indexing and quick compilation of the C# source code.
- **Operating System:** Windows 11 Home 64-bit (10.0, Build 26200 )

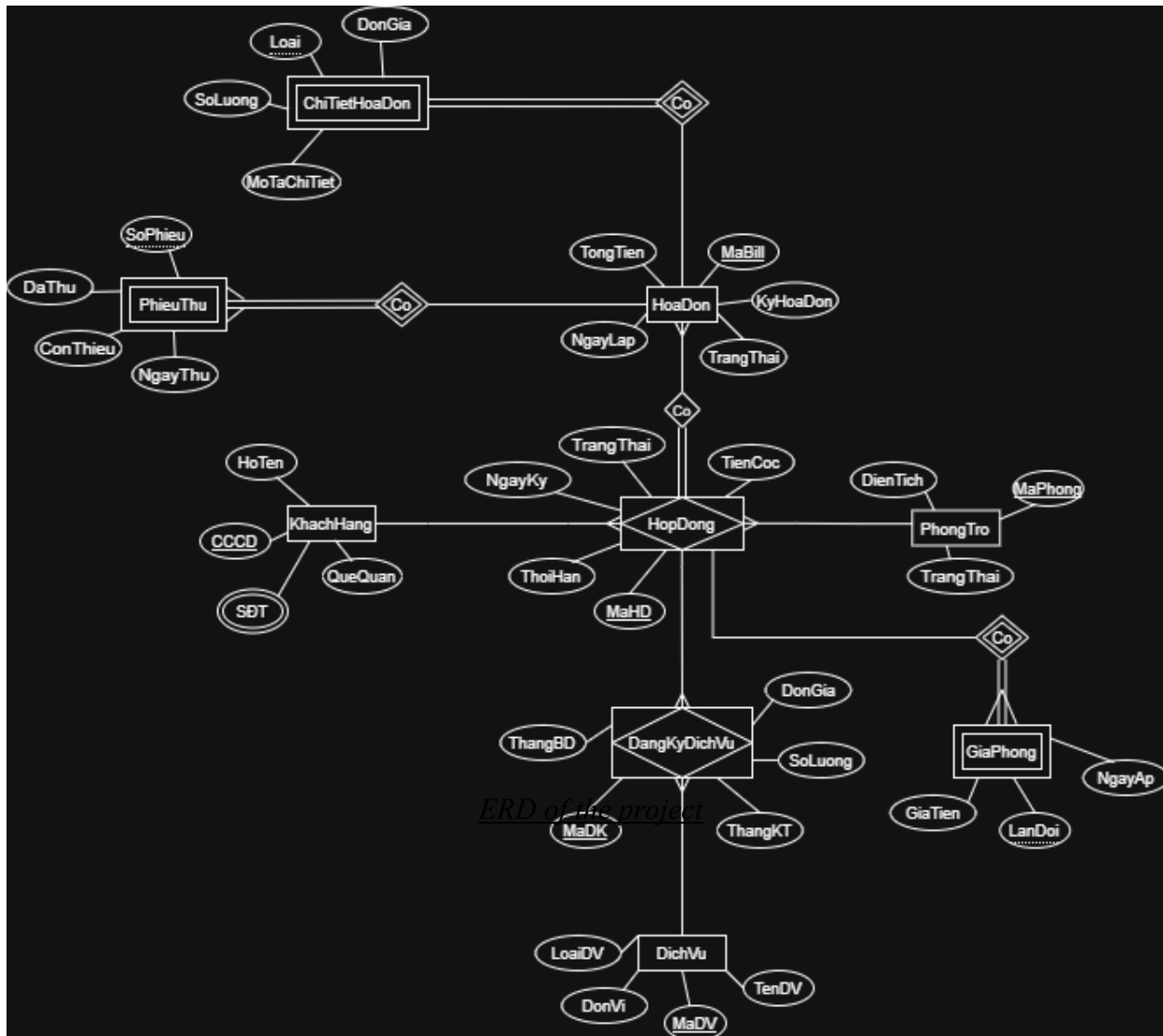
# DATABASE DESIGN OVERVIEW

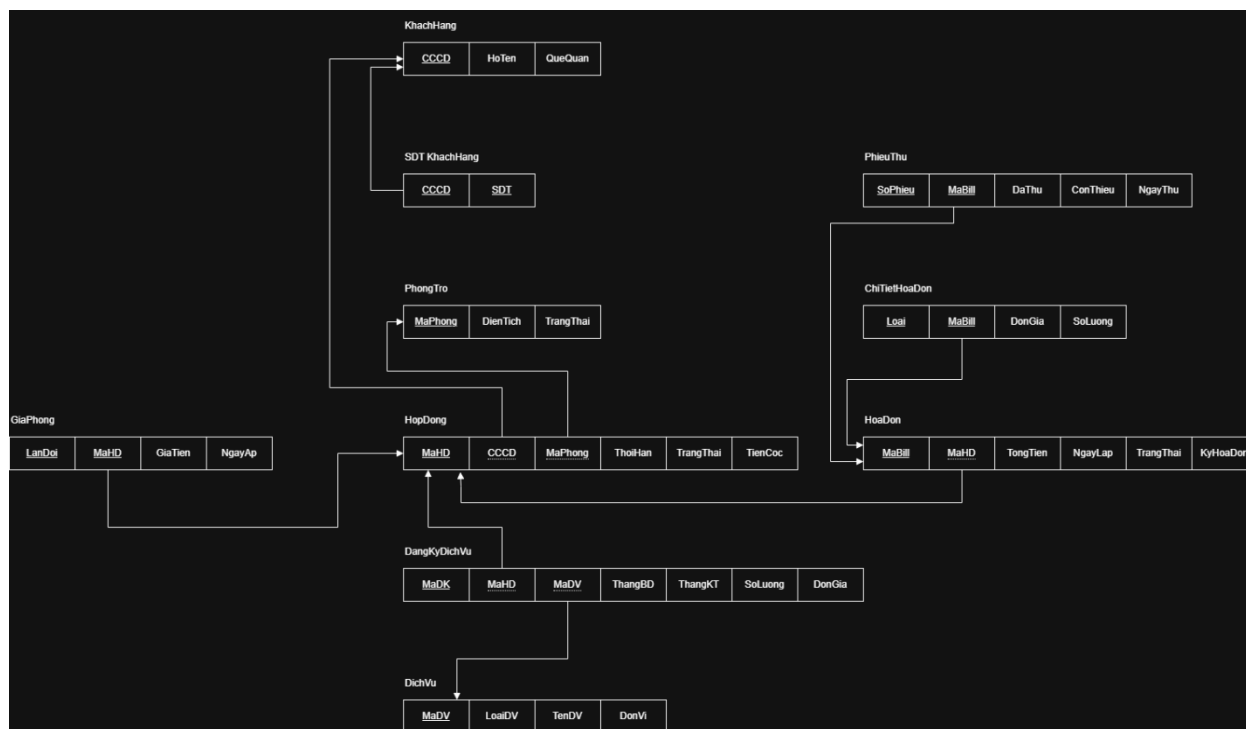
## I. System Overview

The project is designed to automate the administrative tasks of elderly landlords. The system focuses on maintaining high data integrity for long-term rental contracts and monthly financial transactions.

- **Data Consistency:** Using MySQL's InnoDB engine to handle foreign key constraints and ensure that no data is orphaned.
- **Logic Automation:** Shifting complex calculations (like total bills) from the application layer to the database layer using Stored Procedures and Views.
- **Financial Reliability:** Implementing Transactions to guarantee that payment records and debt updates occur as a single, atomic operation.

## II. Entity Relationship Diagram





Schema

### III. Database Modules Summary

#### 1. Customer Management

This is the input module responsible for managing the profiles of individuals renting the rooms.

- **Functions:** Add, edit, and delete tenant information; maintain multiple contact phone numbers.
- **Database Components:** Tables - KhachHang, SDT\_KhachHang.

#### 2. Room Management

This module allows the landlords to keep track of the current status of their “properties”.

- **Functions:** Update room area; monitor statuses such as "Available" or "Occupied".
- **Database Components:** Table - PhongTro

View - *vw\_TinhTrangPhong*.

#### 3. Contract & Service Management

The core module that links Tenants with Rooms.

- **Functions:** Create new rental agreements; register services (electricity, water, internet); set rental prices and security deposits.
- **Database Components:** Tables - HopDong, GiaPhong, DichVu, DangKyDichVu.

Stored Procedures - *sp\_ThemHopDong*, *sp\_HuyHopDong*.

#### 4. Billing & Debt Management

This module handles the most complex computational logic within the system.

- **Functions:** Finalize monthly costs; generate invoices; process payments; track outstanding balances per tenant.
- **Database Components:** Tables - HoaDon, ChiTietHoaDon, PhieuThu

Stored Procedures - *sp\_TaoHoaDon*, *sp\_ThanhToanVaXoaNo*

Trigger *trg\_CapNhatDuNo*.

#### 5. Reporting & Analytics

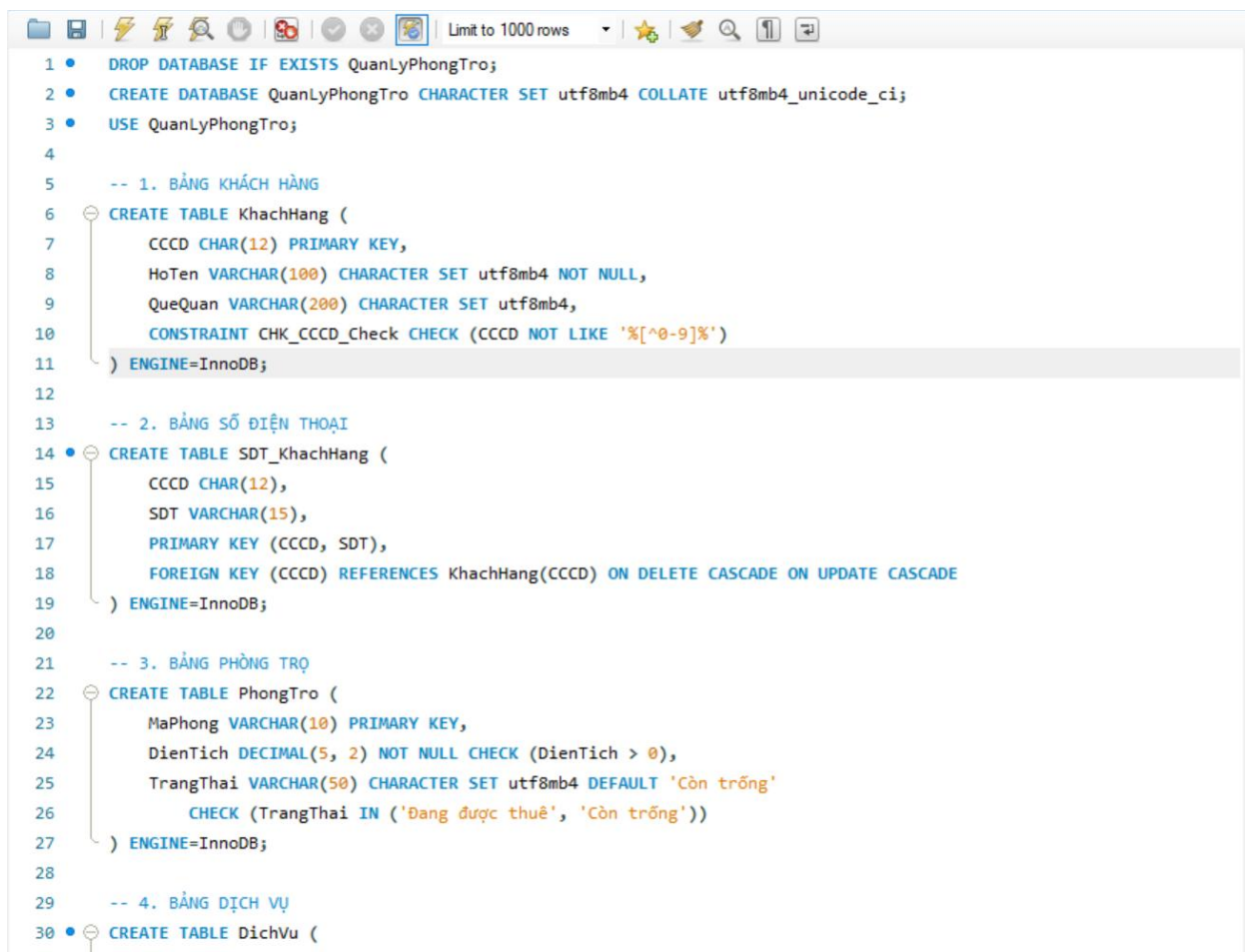
Designed for aggregating and analyzing data over a specific operational period.

- **Functions:** View total monthly revenue; generate lists of debtors; calculate estimated taxes payable.
- **Database Components:** Views - *vw\_NguoiNoTien*, *vw\_DoanhThu*, *vw\_ThueDoanhThu*.

# IMPLEMENTATION AND DEPLOYMENT

## I. Data Definition Language (DDL)

This section presents the SQL scripts used to create the physical structure of the boarding house database. Each table is defined with specific data types and constraints (Primary Key, Foreign Key, Not Null) to ensure data integrity.



```

1 • DROP DATABASE IF EXISTS QuanLyPhongTro;
2 • CREATE DATABASE QuanLyPhongTro CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3 • USE QuanLyPhongTro;
4
5 -- 1. BẢNG KHÁCH HÀNG
6 CREATE TABLE KhachHang (
7     CCCD CHAR(12) PRIMARY KEY,
8     HoTen VARCHAR(100) CHARACTER SET utf8mb4 NOT NULL,
9     QueQuan VARCHAR(200) CHARACTER SET utf8mb4,
10    CONSTRAINT CHK_CCCD_Check CHECK (CCCD NOT LIKE '%[^0-9]%')
11 ) ENGINE=InnoDB;
12
13 -- 2. BẢNG SỐ ĐIỆN THOẠI
14 • CREATE TABLE SDT_KhachHang (
15     CCCD CHAR(12),
16     SDT VARCHAR(15),
17     PRIMARY KEY (CCCD, SDT),
18     FOREIGN KEY (CCCD) REFERENCES KhachHang(CCCD) ON DELETE CASCADE ON UPDATE CASCADE
19 ) ENGINE=InnoDB;
20
21 -- 3. BẢNG PHÒNG TRỌ
22 • CREATE TABLE PhongTro (
23     MaPhong VARCHAR(10) PRIMARY KEY,
24     DienTich DECIMAL(5, 2) NOT NULL CHECK (DienTich > 0),
25     TrangThai VARCHAR(50) CHARACTER SET utf8mb4 DEFAULT 'Còn trống'
26     CHECK (TrangThai IN ('Đang được thuê', 'Còn trống'))
27 ) ENGINE=InnoDB;
28
29 -- 4. BẢNG DỊCH VỤ
30 • CREATE TABLE DichVu (

```

```

30 • CREATE TABLE DichVu (
31     MaDV VARCHAR(10) PRIMARY KEY,
32     LoaiDV VARCHAR(50) CHARACTER SET utf8mb4 NOT NULL
33     CHECK (LoaiDV IN ('Bắt buộc', 'Lựa chọn')),
34     TenDV VARCHAR(100) CHARACTER SET utf8mb4 NOT NULL,
35     DonVi VARCHAR(20) CHARACTER SET utf8mb4
36 ) ENGINE=InnoDB;
37
38 -- 5. BẢNG HỢP ĐỒNG
39 • CREATE TABLE HopDong (
40     MaHD VARCHAR(15) PRIMARY KEY,
41     CCCD CHAR(12) NOT NULL,
42     MaPhong VARCHAR(10) NOT NULL,
43     NgayKy DATE DEFAULT (CURRENT_DATE),
44     ThoiHan INT CHECK (ThoiHan > 0),
45     TienCoc DECIMAL(15, 2) DEFAULT 0 CHECK (TienCoc >= 0),
46     TrangThai VARCHAR(50) CHARACTER SET utf8mb4 NOT NULL
47     CHECK (TrangThai IN ('Còn hiệu lực', 'Hết hiệu lực', 'Bị hủy bỏ')),
48     FOREIGN KEY (CCCD) REFERENCES KhachHang(CCCD) ON DELETE RESTRICT ON UPDATE CASCADE,
49     FOREIGN KEY (MaPhong) REFERENCES PhongTro(MaPhong) ON DELETE RESTRICT ON UPDATE CASCADE
50 ) ENGINE=InnoDB;
51
52 -- 6. BẢNG GIÁ PHÒNG
53 • CREATE TABLE GiaPhong (
54     LanDoi INT AUTO_INCREMENT,
55     MaHD VARCHAR(15),
56     GiaTien DECIMAL(15, 2) NOT NULL CHECK (GiaTien > 0),
57     NgayAp DATE NOT NULL,
58     PRIMARY KEY (LanDoi, MaHD),
59     FOREIGN KEY (MaHD) REFERENCES HopDong(MaHD) ON DELETE CASCADE ON UPDATE CASCADE
60 ) ENGINE=InnoDB;
61
62 -- 7. BẢNG ĐĂNG KÝ DỊCH VỤ
63 • CREATE TABLE DangKyDichVu (
64     MaDK INT AUTO_INCREMENT PRIMARY KEY,
65     MaHD VARCHAR(15) NOT NULL,
66     MaDV VARCHAR(10) NOT NULL,
67     ThangBD DATE,
68     ThangKT DATE,
69     SoLuong INT DEFAULT 1 CHECK (SoLuong > 0),
70     DonGia DECIMAL(15, 2) CHECK (DonGia >= 0),
71     FOREIGN KEY (MaHD) REFERENCES HopDong(MaHD) ON DELETE CASCADE,
72     FOREIGN KEY (MaDV) REFERENCES DichVu(MaDV) ON DELETE RESTRICT
73 ) ENGINE=InnoDB;
74
75 -- 8. BẢNG HÓA ĐƠN
76 • CREATE TABLE HoaDon (
77     MaBill VARCHAR(20) PRIMARY KEY,
78     MaHD VARCHAR(15) NOT NULL,
79     TongTien DECIMAL(15, 2) DEFAULT 0 CHECK (TongTien >= 0),
80     NgayLap DATE DEFAULT (CURRENT_DATE),
81     TrangThai VARCHAR(50) CHARACTER SET utf8mb4 DEFAULT 'Còn thiếu'
82     CHECK (TrangThai IN ('Đã thanh toán đủ', 'Còn thiếu')),
83     KyHoaDon VARCHAR(10),
84     FOREIGN KEY (MaHD) REFERENCES HopDong(MaHD) ON DELETE RESTRICT ON UPDATE CASCADE
85 ) ENGINE=InnoDB;
86
87 -- 9. BẢNG CHI TIẾT HÓA ĐƠN
88 • CREATE TABLE ChiTietHoaDon (
89     MaBill VARCHAR(20),

```

```
90     Loai VARCHAR(50) CHARACTER SET utf8mb4 CHECK (Loai IN ('TienPhong', 'TienDichVu')),
91     MoTaChiTiet VARCHAR(255) CHARACTER SET utf8mb4,
92     DonGia DECIMAL(15, 2) CHECK (DonGia >= 0),
93     SoLuong INT CHECK (SoLuong >= 0),
94     PRIMARY KEY (MaBill, Loai),
95     FOREIGN KEY (MaBill) REFERENCES HoaDon(MaBill) ON DELETE CASCADE ON UPDATE CASCADE
96 ) ENGINE=InnoDB;
97
98 -- 10. BẢNG PHIẾU THU
99 ● ○ CREATE TABLE PhieuThu (
100     SoPhieu INT AUTO_INCREMENT PRIMARY KEY,
101     MaBill VARCHAR(20) NOT NULL,
102     DaThu DECIMAL(15, 2) DEFAULT 0 CHECK (DaThu >= 0),
103     ConThieu DECIMAL(15, 2),
104     NgayThu DATETIME DEFAULT CURRENT_TIMESTAMP,
105     FOREIGN KEY (MaBill) REFERENCES HoaDon(MaBill) ON DELETE RESTRICT ON UPDATE CASCADE
106 ) ENGINE=InnoDB;
```



## II. Advanced Database Logic (Stored Procedure)

Procedures are used to handle multi-step processes, such as adding a new contract or generating monthly bills, ensuring that all related tables are updated consistently.

### 1. Add New Contract (Automatically Update Room Status)

**Description:** This procedure automates the rental process by creating a contract record and simultaneously changing the room status to "Occupied".

```
CREATE PROCEDURE sp_ThemHopDong
(
    IN p_MaHD VARCHAR(15), IN p_CCCD CHAR(12),
    IN p_MaPhong VARCHAR(10), IN p_ThoiHan INT, IN p_TienCoc DECIMAL(15,2)
)
BEGIN
    INSERT INTO HopDong(MaHD, CCCD, MaPhong, ThoiHan, TienCoc, TrangThai)
    VALUES (p_MaHD, p_CCCD, p_MaPhong, p_ThoiHan, p_TienCoc, 'Còn hiệu lực');

    UPDATE PhongTro SET TrangThai = 'Đang được thuê' WHERE MaPhong = p_MaPhong;
END
```

### 2. Cancel Contract (Immediate Room Release)

**Description:** Handles the termination of a contract and reverts the room status to "Available" for new tenants.

```
CREATE PROCEDURE sp_HuyHopDong(IN p_MaHD VARCHAR(15))
BEGIN
    DECLARE v_MaPhong VARCHAR(10);
    SELECT MaPhong INTO v_MaPhong FROM HopDong WHERE MaHD = p_MaHD;
```

```
UPDATE HopDong SET TrangThai = 'Bị hủy bỏ' WHERE MaHD = p_MaHD;
UPDATE PhongTro SET TrangThai = 'Còn trống' WHERE MaPhong = v_MaPhong;
END
```

### 3. Generate Invoice (Automated Rent and Service Consolidation)

**Description:** Automatically calculates the total monthly amount by fetching fixed rent prices and summing up all registered service costs.

```
CREATE PROCEDURE sp_TaoHoaDon(IN p_MaBill VARCHAR(20), IN p_MaHD
VARCHAR(15), IN p_KyHoaDon VARCHAR(10))
BEGIN
    DECLARE v_TienPhong DECIMAL(15,2);
    DECLARE v_TienDichVu DECIMAL(15,2);
    DECLARE v_TongTien DECIMAL(15,2);
    SELECT GiaTien INTO v_TienPhong FROM GiaPhong
    WHERE MaHD = p_MaHD ORDER BY NgayAp DESC LIMIT 1;
    SELECT COALESCE(SUM(SoLuong * DonGia), 0) INTO v_TienDichVu
    FROM DangKyDichVu WHERE MaHD = p_MaHD;
    SET v_TongTien = v_TienPhong + v_TienDichVu;

    INSERT INTO HoaDon(MaBill, MaHD, TongTien, KyHoaDon, TrangThai)
    VALUES (p_MaBill, p_MaHD, v_TongTien, p_KyHoaDon, 'Còn thiếu');

    INSERT INTO ChiTietHoaDon(MaBill, Loai, MoTaChiTiet, DonGia, SoLuong)
    VALUES (p_MaBill, 'TienPhong', 'Tiền thuê phòng', v_TienPhong, 1);
END
```

### III. Data Querying (DQL) & Views

To simplify complex reporting for the landlord, Database Views are created to aggregate revenue and debt information without writing complex JOIN queries repeatedly.

#### 1. Active Tenant View

**Description:**List of current customers currently renting

```
CREATE OR REPLACE VIEW vw_NguoiThue AS
SELECT k.CCCD, k.HoTen, k.QueQuan, h.MaPhong, h.NgayKy, h.TrangThai
FROM KhachHang k
JOIN HopDong h ON k.CCCD = h.CCCD
WHERE h.TrangThai = 'Còn hiệu lực';
```

#### 2. Room Status View

**Description:**Current availability and condition of all rooms

```
CREATE OR REPLACE VIEW vw_TinhTrangPhong AS
SELECT MaPhong, DienTich, TrangThai
FROM PhongTro;
```

#### 3. Contract List View

**Description:**A comprehensive overview of all signed rental agreements

```
CREATE OR REPLACE VIEW vw_DanhSachHopDong AS
SELECT h.MaHD, k.HoTen, h.MaPhong, h.NgayKy, h.ThoiHan, h.TienCoc, h.TrangThai
FROM HopDong h
```

```
JOIN KhachHang k ON h.CCCD = k.CCCD;
```

#### 4. Debtor List View

**Description:**Specifically filters tenants who have outstanding balances

```
CREATE OR REPLACE VIEW vw_NguoiNoTien AS
SELECT hd.MaBill, k.HoTen, p.MaPhong, hd.TongTien,
       COALESCE(SUM(pt.DaThu), 0) AS DaThanhToan,
       (hd.TongTien - COALESCE(SUM(pt.DaThu), 0)) AS ConNo
FROM HoaDon hd
JOIN HopDong h ON hd.MaHD = h.MaHD
JOIN KhachHang k ON h.CCCD = k.CCCD
JOIN PhongTro p ON h.MaPhong = p.MaPhong
LEFT JOIN PhieuThu pt ON hd.MaBill = pt.MaBill
GROUP BY hd.MaBill, k.HoTen, p.MaPhong, hd.TongTien
HAVING ConNo > 0;
```

#### 5. Revenue by Billing Cycle View

**Description:**Summarizes total income based on specific billing periods

```
CREATE OR REPLACE VIEW vw_DoanhThu AS
SELECT KyHoaDon, SUM(DaThu) AS TongDoanhThuThucTe
FROM PhieuThu pt
JOIN HoaDon hd ON pt.MaBill = hd.MaBill
```

```
GROUP BY KyHoaDon;
```

## 6. Tax Estimation View (Based on 5% Revenue)

**Description:**Calculates projected tax obligations assuming a 5% rate

```
CREATE OR REPLACE VIEW vw_ThueDoanhThu AS
SELECT KyHoaDon, TongDoanhThuThucTe,
       (TongDoanhThuThucTe * 0.05) AS ThuePhaiNop
FROM vw_DoanhThu;
```

## IV. Integrity and Consistency

### 1. Triggers

Triggers are implemented to automate data updates. For instance, when a payment is recorded, a trigger automatically updates the remaining debt in the Invoice table

**Update Debt Balance:** Automatically adjusts the outstanding balance when a tenant requests a payment deferral or fails to pay in full

```
DELIMITER //
CREATE TRIGGER trg_CapNhatDuNo
BEFORE INSERT ON PhieuThu
FOR EACH ROW
BEGIN
    DECLARE v_TongTien DECIMAL(15,2);
    DECLARE v_DaThuTruocDo DECIMAL(15,2);
    SELECT TongTien INTO v_TongTien FROM HoaDon WHERE MaBill = NEW.MaBill;
    SELECT COALESCE(SUM(DaThu), 0) INTO v_DaThuTruocDo
    FROM PhieuThu WHERE MaBill = NEW.MaBill;
    SET NEW.ConThieu = v_TongTien - v_DaThuTruocDo - NEW.DaThu;
    IF NEW.ConThieu < 0 THEN
        SET NEW.ConThieu = 0;
    END IF;
END //
DELIMITER ;
```

### 2. Transaction

Ensures that recording a receipt and deducting from the debt balance occur as a single atomic operation to prevent data loss

**Payment & Debt Clearance:** Ensures that recording a receipt and deducting from the debt balance occur as a single atomic operation to prevent data loss.

```
DELIMITER //

CREATE PROCEDURE sp_ThanhToanVaXoaNo
(
    IN p_MaBill VARCHAR(20),
    IN p_SoTienNop DECIMAL(15,2)
)
BEGIN
    DECLARE v_ConThieu DECIMAL(15,2);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
    END;

    START TRANSACTION;
    INSERT INTO PhieuThu(MaBill, DaThu) VALUES (p_MaBill, p_SoTienNop);
    SELECT ConThieu INTO v_ConThieu FROM PhieuThu
    WHERE MaBill = p_MaBill ORDER BY SoPhieu DESC LIMIT 1;
    IF v_ConThieu <= 0 THEN
        UPDATE HoaDon SET TrangThai = 'Đã thanh toán đủ' WHERE MaBill = p_MaBill;
    END IF;
    COMMIT;
END //

DELIMITER ;
```

# CONCLUSION

## I. Summary

The project has successfully developed a "Boarding House Management System" that meets the initial requirements for efficiency and data integrity. By leveraging MySQL and C# (WPF), the system provides a reliable solution for landlords to manage tenants, rooms, and financial transactions without the risk of manual errors.



## II. Key Achievements

- **Database Mastering:** Successfully implemented advanced logic such as Stored Procedures for automated billing and Triggers for real-time debt updates.
- **Financial Integrity:** Ensured safe financial processing through the use of Transactions, preventing data loss during payment records.
- **User Experience:** Designed a simplified interface tailored for elderly landlords, making technology more accessible.

### III. Lessons Learned

- **Complex Data Relationships:** Managing the 1:N relationship between tenants and multiple phone numbers while ensuring the HopDong (Contract) table remains the central source of truth.
- **Financial Logic Accuracy:** Designing Stored Procedures that correctly aggregate various services (electricity, water) and fixed rent without double-counting.
- **Concurrency Control:** Ensuring that when multiple payments are made, the database avoids "dirty reads" or lost updates using appropriate Transaction levels.
- **Database First Approach:** Learned the importance of a solid ERD design before writing any code; a well-structured database schema significantly reduces backend bugs.
- **Power of Automation:** Discovered how Triggers and Procedures can simplify application code by handling complex business rules directly within MySQL.
- **Data Integrity over UI:** Realized that while the interface (WPF) is important for the elderly, the integrity of the data (ACID) is the backbone of any management system

## IV. Future Work

Although the system is functional, there are several areas for future improvement to enhance its capabilities:

- **Cloud Integration:** Migrating the database to a cloud-based server (e.g., Azure or AWS) to allow the landlord to manage the property remotely via mobile devices.
- **Payment Gateway Integration:** Integrating with e-wallets like MoMo or VNPay to automate the payment reconciliation process.
- **IoT Smart Metering:** Connecting with smart electricity and water meters to eliminate manual data entry.

## REFERENCES

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.
2. Garcia-Molina, H., Ullman, J. D., & Widom, J. (2008). *Database Systems: The Complete Book* (2nd ed.). Pearson.
3. Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
4. Date, C. J. (2003). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.
5. Connolly, T., & Begg, C. (2014). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson.
6. Ramakrishnan, R., & Gehrke, J. (2002). *Database Management Systems* (3rd ed.). McGraw-Hill.
7. W3C Web Accessibility Initiative (WAI). (2026). *Web Content Accessibility Guidelines (WCAG) 2.1* (Essential for elderly-friendly UI design).
8. Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann (Standard for user-friendly interface evaluation).
9. Oracle Corporation. (2026). *MySQL 8.0 Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/>.
10. Microsoft Learn. (2026). *C# Language Documentation*. [Online]. Available: <https://learn.microsoft.com/dotnet/csharp/>.
11. Microsoft Learn. (2026). *Windows Presentation Foundation (WPF) Documentation*. [Online]. Available: <https://learn.microsoft.com/dotnet/desktop/wpf/>.
12. MySQL Connector/NET Developer Guide. *Connecting C# to MySQL*. Oracle Documentation, 2026.
13. Entity Framework Core Documentation. *Object-Relational Mapping for .NET*. Microsoft, 2026.
14. Material Design in XAML Toolkit. *Modern UI Components for WPF*. [Online]. Available: <http://materialdesigninxaml.net/>.
15. Skeet, J. (2019). *C# in Depth* (4th ed.). Manning Publications.